

```
[5]: #Mean
# it is calculated by adding all val and divided by the total no of val . Formula mean=sum of all val/total no of val
marks=[45,50,55,60,90]
mean = sum(marks)/ len(marks)
print("Mean:",mean)

Mean: 60.0
```

```
[7]: #Median
# it is only find by after sorting the data
#1. Arrange data in asc order
#2. find the mid val

#when to use median?
#when data contains outliers
#Example:
#salary=[20000,22000,25000,28000,50000]
#mean becomes very large bcz of 50000
#median remains
#25000
#so median gives a better representation
marks=[45,50,55,60,90]
median=statistics.median(marks)
print("Median",median)

Median 55
```

```
[8]: # MODE
#it is the val that appears most frequently
#Example
#data=[10,20,30,40]
#here:
#20 appears 2 times
```

```
#20 appears 2 times
#so mode =20
import statistics
data=[10,20,20,30,40]
mode=statistics.mode(data)
print("Mode",mode)
```

Mode 20

```
[12]: #when to use
#finding the most common item
#Ex:
#most common shirt size
```

```
import pandas as pd
df=pd.DataFrame({
    'Marks':[45,50,55,60,90]
})
print("Mean:")
print(df['Marks'].mean())
print("Median:")
print(df['Marks'].median())
print("Mode:")
print(df['Marks'].mode())
```

Mean:

60.0

Median:

55.0

Mode:

0 45

1 50

2 55

3 60

4 90

Name: Marks, dtype: int64

```
5 00
4 90
```

Name: Marks, dtype: int64

Variance & Standard Deviation-Measure of Speed

```
[13]: #Measures of spread tell us how far data val are spread from the center(mean)
      #Two datasets can have the same mean but very diff spread
      #Ex: a=[48,49,50,51,52]
      #b=[20,35,50,65,80]
      #Both have mean=50
      #but dataset b has much more spread out
```

#variance

#It measures how far the data val are from the mean or avg

14.0

Variance: 8.0

```
[14]: import numpy as np
      data=[10,12,14,16,18]
      print(np.mean(data))
      variance=np.var(data)
      print("Variance:",variance)
```

14.0

Variance: 8.0

```
[ ]: #Standard deviation
      #SD is the square root of variance.
      # Interpretation
      # .Small SD - Data points are close to mean.
      # .Large SD - Data points are far from mean.
```

```
[15]: import numpy as np
data=[10,12,14,16,18]
print(np.mean(data))
variance=np.var(data)
std=np.std(data)
print("Variance:",variance)
print("std:",std)
```

14.0

Variance: 8.0

std: 2.8284271247461903

```
[ ]: #Interquartile range(IQR)
# IQR measures the spread of the middle 50% of data
# It is less affected by outliers than variance and SD
#FORMULA
# IQR=Q3-Q1

#Where:
# Q1=First Quartile(25%)
#Q3=Third Quartile(75%)

#Ex: data=[5,10,15,20,25,30,35,40]
#Quartiles
Q1=13.75
Q3=31.25
IQR
IQR=31.25-13.75
```

```
[16]: import numpy as np
data=[5,10,15,20,25,30,35,40]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
```

```
data=[10,12,14,15,16,18,100]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR=Q3-Q1
print("Q1:",Q1)
print("Q3:",Q3)
print("IQR:",IQR)
```

```
Q1: 13.75
Q3: 31.25
IQR: 17.5
```

```
[ ]: #IQR for Outlier Detection
Outliers are often detected using:
lower limit:Q1-1.5*IQR
upper limit:Q3+1.5*IQR
Any value outside these limits is considered an outlier
```

```
[18]: # Code
import numpy as np
data=[10,12,14,15,16,18,100]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR=Q3-Q1
lower=Q1-1.5*IQR
upper=Q3+1.5*IQR
print(lower)
print(upper)
outliers=[x for x in data if x < lower or x > upper]
print("Outliers:",outliers)

7.0
23.0
Outliers: [100]
```

```
[ ]: #Probability basics
probability is the measure of how likely an event is to occur
0=impossible event
1=certain event

example
probability of getting head in a coin toss=0.5
probability of rolling a 6 on a die = 1/6

# 1.sample space(s)
The set of all possible outcomes of an experiment.
# 2.Event(A)
An event is a subset of the sample space.

#3. Probability of an Event P(A)
Example
Find probability of getting an even number from a die.
sample_space=[1,2,3,4,5,6]
event_A=[2,4,6]
P_A=len(event_A)/len(sample_space)
print("P(A)=",P_A)

#4.Conditional probability P(A|B)
probability of A occurring given that B has already occurred.

#5. Bayes' Theorem
Used to calculate the probability of an event based on prior knowledge.

    Example: Disease Testing
    Suppose:

    P_Disease = 0.01
```

Suppose:

```
P_Disease = 0.01
P_Positive_given_Disease=0.95
P_Positive=0.05
```

```
[19]: sample_space={1,2,3,4,5,6}
      event_A={2,4,6} # even numbers
      print("Sample space (S):",sample_space)
      print("Event A:",event_A)

      #2. Probability P(A)
      P_A = len(event_A) / len(sample_space)
      print("\nProbability of Event A")
      print("P(A)=",P_A)

      #3. conditional probability
      #P(A|B)

      event_B = {4,5,6}
      intersection = event_A.intersection(event_B)
      P_A_given_B = len(intersection) / len(event_B)
      print("\nEvent B:",event_B)
      print("A ∩ B:",intersection)
      print("P(A|B)=",P_A_given_B)

      Sample space (S): {1, 2, 3, 4, 5, 6}
      Event A: {2, 4, 6}

      Probability of Event A
      P(A)= 0.5

      Event B: {4, 5, 6}
      A ∩ B: {4, 6}
```

```
[21]: #Bayes Theorem
#Example:
#Disease prevalence=1%
#Test sensitivity =99%
#positive test probability = 5%

P_Disease=0.01
P_Positive_given_Disease=0.99
P_Positive = 0.05

P_Disease_given_Positive=(
    P_Positive_given_Disease*P_Disease
) / P_Positive

print("\nBayes' Theorem Example")
print("P(Disease) =",P_Disease)
print("P(Positive|Disease) =",P_Positive_given_Disease)
print("P(Positive) =",P_Positive)

print(
    "P(Disease|Positive) =",
    round(P_Disease_given_Positive,4)
)

print(
    "Percentage=",
    round(P_Disease_given_Positive*100,2),
    "%"
)
```

```
Bayes' Theorem Example
P(Disease) = 0.01
P(Positive|Disease) = 0.99
P(Positive) = 0.05
P(Disease|Positive) = 0.198
Percentage= 19.8 %
```



```
[ ]: #CORRELATION
```

What **is** correlation?

correlation measures the relationship

Study hours (increase) - marks(increase)

#Pearson correlation

Measures linear relationship between two numerical variables

When to use pearson?

Numerical data

linear relationships

less outliers

Ex:

height & weight

study hours & marks

advertising spend **and** sales

```
[ ]: import pandas as pd
df=pd.DataFrame({
    'Hours': [1,2,3,4,5],
    'Marks': [20,40,50,70,90]
})
corr = df['Hours'].corr(df['Marks'])
print("Pearson Correlation:",corr)
```

#close to +1

```
[ ]: #spearman correlation
Measures rank based (monotonic) relationships.
Used when:
data is not normally distributed
outliers exist
Relationship is not perfectly linear
```

```
[23]: import pandas as pd
df = pd.DataFrame({
    'Rank_Study': [1,2,3,4,5],
    'Rank_Marks': [2,1,3,5,4]
})
corr = df['Rank_Study'].corr(
    df['Rank_Marks'],
    method='spearman'
)
print("Spearman Correlation:",corr)

Spearman Correlation: 0.7999999999999999
```

```
[24]: #Correlation matrix
import pandas as pd
df = pd.DataFrame({
    'Hours': [1,2,3,4,5],
    'Marks': [23,40,50,70,90],
    'Sleep': [8,7,6,5,4]
})
print(df.corr())
```

	Hours	Marks	Sleep
Hours	1.000000	0.993812	-1.000000
Marks	0.993812	1.000000	-0.993812
Sleep	-1.000000	-0.993812	1.000000

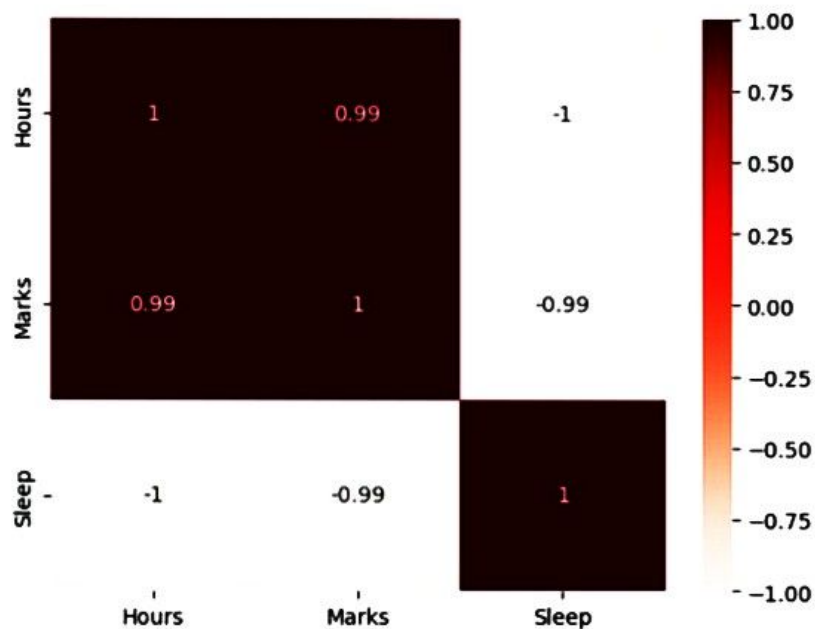
```
[ ]: # Interpretation
Hours vs Marks = 0.99
strong negative correlation
```

```
[31]: #Heatmap for correlation
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df=pd.DataFrame({
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,70,90],
    'Sleep':[8,7,6,5,4]
})
sns.heatmap(df.corr(),
    annot=True,
    cmap='Reds')
plt.show()

#Common cmap values
#coolwarm
#viridis
#YlGnBu
#Blues
#Reds
#greens
#magma
```



#magma



```
[32]: import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

#Generate normally distribution data
data= np.random.normal(
    loc=70, #mean
    scale=10, #standard deviation
    size=1000 #number of values
```

Hours

Marks

Sleep

--1.00

```
[32]: import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

#Generate normally distribution data
data= np.random.normal(
    loc=70, #mean
    scale=10, #standard deviation
    size=1000 #number of values
)
# calculate mean and standard deviation
mean=np.mean(data)
std=np.std(data)

print("Mean:",round(mean,2))
print("Standard Deviation:",round(std,2))

#plot histogram
plt.figure(figsize=(8,5))
sns.histplot(data,bins=30,kde=True)

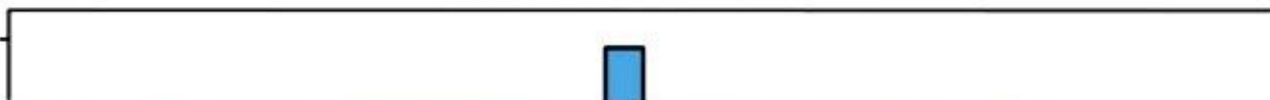
plt.title("Normal Distribution")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```

Mean: 70.07

Standard Deviation: 10.13

Normal Distribution

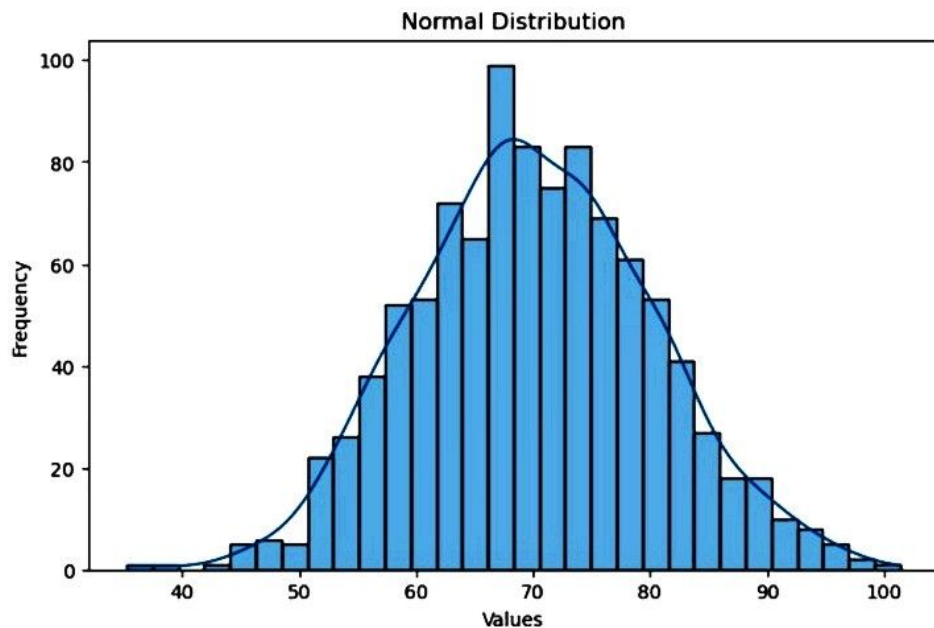
100



```
plt.show()
```

Mean: 70.07

Standard Deviation: 10.13



```
[ ]: #Right-skewed(Positive skew)
long tail on the right
mean>median
example: Income data
```

values

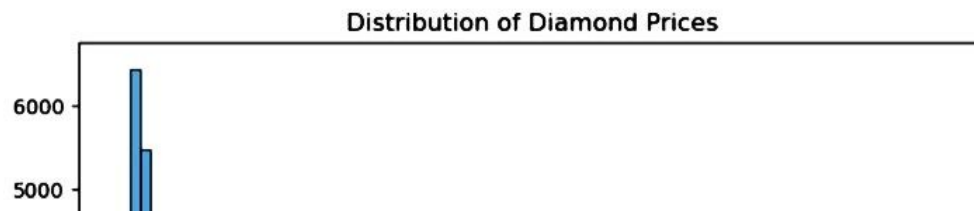
```
[ ]: #Right-skewed(Positive skew)
long tail on the right
mean>median
example: Income data
```

```
#Left-skewed(Negative skew)
long tail on the left
mean<median
example:Easy exam scores
```

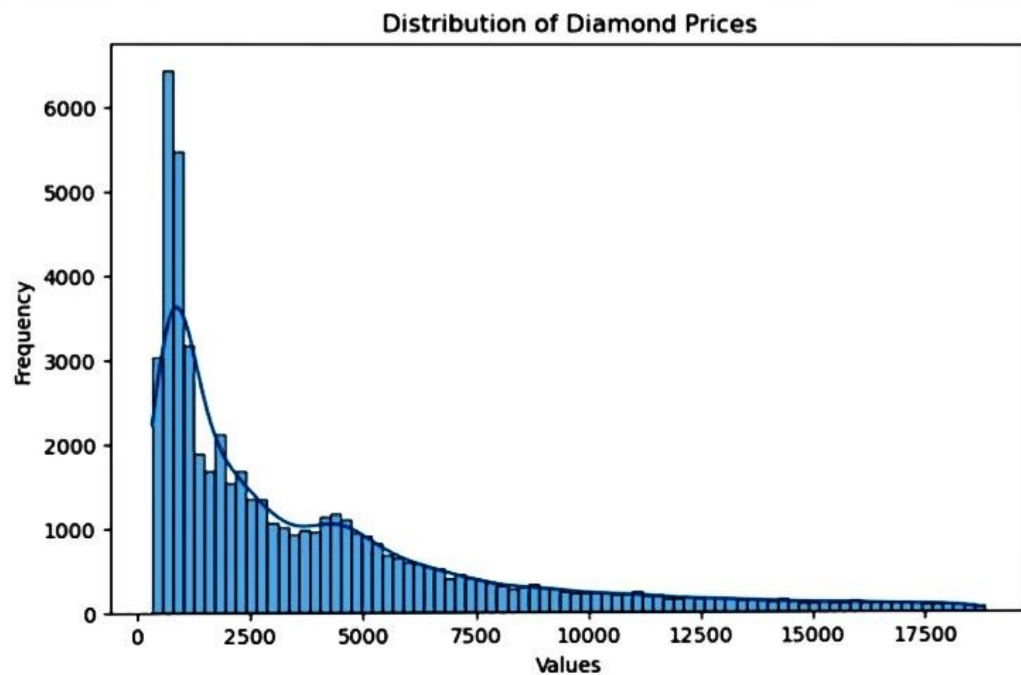
```
[35]: import seaborn as sns
import matplotlib.pyplot as plt
```

```
#Load dataset
df = sns.load_dataset("diamonds")

#Histogram + KDE
plt.figure(figsize=(8,5))
sns.histplot(df["price"],kde=True)
plt.title("Distribution of Diamond Prices")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```



```
plt.show()
```



```
[ ]:
```

```
[38]: import seaborn as sns
import matplotlib.pyplot as plt
```


0 2500 5000 7500 10000 12500 15000 17500
Values

[]:

```
[38]: import seaborn as sns
import matplotlib.pyplot as plt

#Load dataset
df = sns.load_dataset("diamonds")

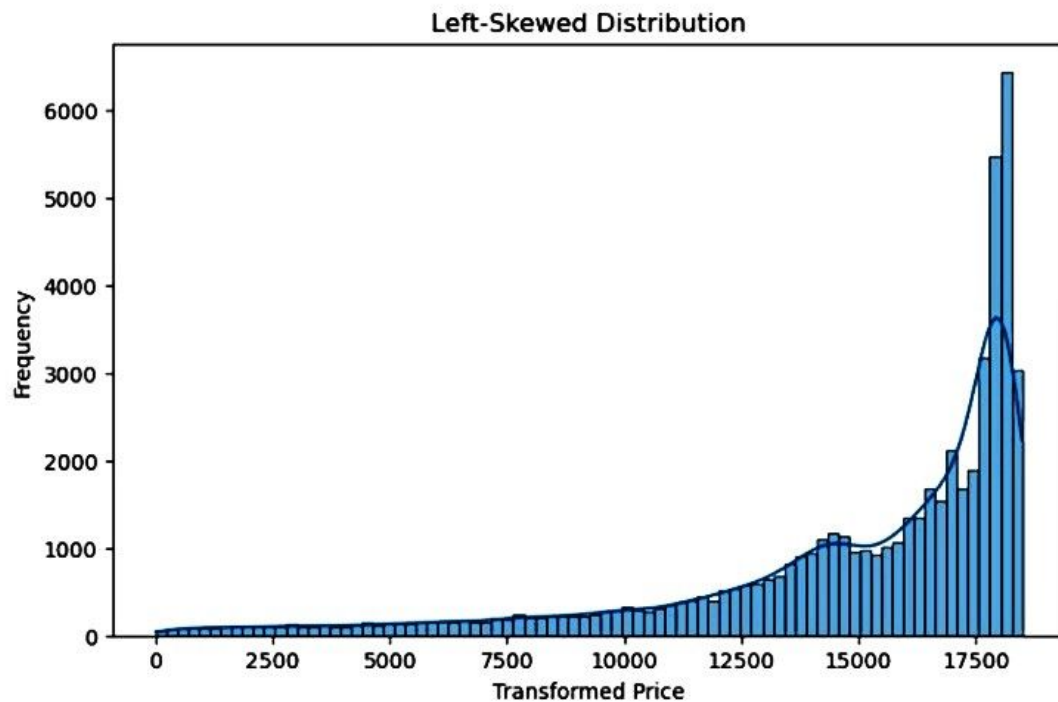
#Create left-skewed data
left_skew = df["price"].max() - df["price"]

#plot
plt.figure(figsize=(8,5))
sns.histplot(left_skew,kde=True)
plt.title("Left-Skewed Distribution")
plt.xlabel("Transformed Price")
plt.ylabel("Frequency")
plt.show()
```

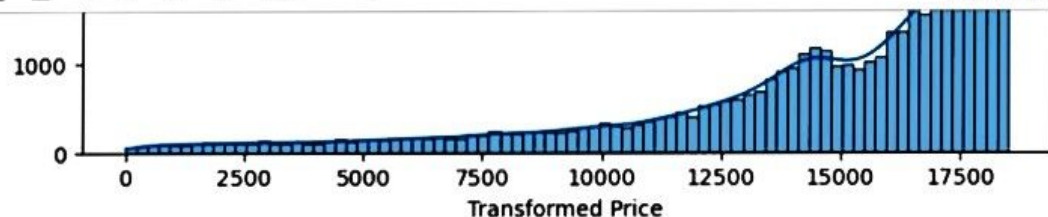
Left-Skewed Distribution



```
plt.xlabel("Transformed Price")  
plt.ylabel("Frequency")  
plt.show()
```



```
[40]: import numpy as np  
scores=[60,65,70,75,80]  
mean=np.mean(scores)  
std=np.std(scores)
```



```
[40]: import numpy as np
scores=[60,65,70,75,80]
mean=np.mean(scores)
std=np.std(scores)

score=80

z_score=(score-mean)/std

print("Mean:",mean)
print("Standard Deviation:",std)
print("Z-score:",z_score)
```

```
Mean: 70.0
Standard Deviation: 7.0710678118654755
Z-score: 1.414213562373095
```

```
[ ]:
```